UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Córdoba

Ingeniería y Calidad de Software

TRABAJO PRÁCTICO N.º 6

Justificación de las decisiones de diseño

EcoHarmony Park

Historia de usuario: Comprar entradas

Curso 4K2 · Grupo G5 · 2026

Integrantes del grupo	Legajo
Baolino, Emily Belén	400850
Brageda, Rocío	93868
Galanti, Matías	86624
Galindo, Facundo	80835
González, Alessandro	92950
Maldonado, Leandro	74553
Mandrilli, Bautista	89654
Massini Arch, Maximiliano	94288
Pedernera, Nicolás	96746
Saggiorato, Gina	95794
Tapia, Belén	87368
Villane, Ignacio	62687
Zitelli, Gabriel Luciano	82187

Índice

1. Tecnologías y presentación 3
1.1. Python y FastAPI 3
1.2. Jinja2 y la interfaz web 3
2. Organización de la lógica de compra 3
2.1. Separación de responsabilidades 3
2.2. Reglas de negocio independientes 3
2.3. Coordinación mediante un caso de uso 4
3. Diseño orientado a las pruebas 4
3.1. Dependencias sustituibles y pytest 4
4. Datos, autenticación y notificaciones 4
4.1. Persistencia local con SQLite 4
4.2. Contraseñas y gestión de sesiones 4
4.3. Confirmaciones mediante SMTP 5
5. Alcance del pago con Mercado Pago 5
6. Convenciones de escritura del código 5

1. Tecnologías y presentación

1.1. Python y FastAPI

Para desarrollar la funcionalidad de compra se eligió Python como lenguaje principal y FastAPI como marco de trabajo para exponer las rutas de la aplicación. Esta combinación responde a un sistema web de alcance acotado, en el que conviven formularios, validaciones y operaciones que deben poder comprobarse de manera aislada. Uvicorn se utiliza para ejecutar la aplicación como servidor ASGI.

En las rutas, FastAPI permite expresar los parámetros esperados y resolver dependencias de forma explícita. Al mismo tiempo, las reglas de compra permanecen escritas como funciones de Python, sin requerir solicitudes HTTP para ejecutarse. Esta distinción resulta particularmente útil en el TP6: los comportamientos del dominio pueden probarse con rapidez, mientras que el recorrido web se verifica por separado con TestClient.

1.2. Jinja2 y la interfaz web

La interfaz se resuelve mediante plantillas Jinja2, complementadas con HTML, CSS y JavaScript. Las pantallas de registro, ingreso y compra comparten una base visual, mientras que cada vista conserva los campos y resultados propios de su función. Esta solución mantiene la presentación fuera de las rutas y evita incorporar marcado HTML dentro de la lógica de aplicación.

JavaScript se reserva para respuestas inmediatas del formulario, como generar un campo de edad por cada visitante y facilitar la elección de fecha mediante Flatpickr. Se trata de ayudas a la interacción y no de la única barrera de validación: el servidor vuelve a comprobar los datos recibidos. La decisión mejora la experiencia de uso sin trasladar las reglas de negocio a un entorno que el usuario puede modificar.

2. Organización de la lógica de compra

2.1. Separación de responsabilidades

El proyecto distribuye su código en dominio, aplicación, infraestructura e interfaces. Esta organización no se adopta como un fin en sí mismo, sino para evitar que una misma pieza de software deba conocer a la vez las reglas de compra, el protocolo HTTP, el acceso a usuarios y el envío de mensajes. Cada parte concentra una responsabilidad reconocible y puede evolucionar con menor interferencia sobre las demás.

Así, los cambios de presentación se localizan principalmente en las interfaces, mientras que la infraestructura reúne los detalles técnicos de servicios externos y persistencia. La lógica central depende menos de tecnologías particulares. En el estado revisado corresponde describirla como una organización por capas de responsabilidad, sin atribuirle formalmente un patrón arquitectónico más estricto que el que demuestra el código.

2.2. Reglas de negocio independientes

Las comprobaciones de cantidad, edades, fecha, tipo de pase y cálculo del importe se concentran en reglas_compra.py. En lugar de incorporarlas a una ruta o plantilla, cada función aborda una condición identificable y trabaja con valores ordinarios de Python. De esa forma, un caso inválido puede comprobarse sin levantar el servidor ni preparar una base de datos.

La posibilidad de indicar una fecha de referencia o un conjunto de días abiertos ayuda a construir pruebas deterministas. Es especialmente valiosa para analizar límites —por ejemplo, fechas anteriores a

la actual o cantidades superiores a diez— sin que el resultado dependa del día en que se ejecuta la suite. La separación, además, deja un lugar claro para modificar una regla cuando el dominio lo requiera.

2.3. Coordinación mediante un caso de uso

La operación comprar_entradas reúne la secuencia que da sentido a la historia de usuario: identificar al comprador, validar los datos, calcular el total, construir el resultado e intentar la notificación correspondiente. Esa coordinación pertenece a la aplicación y no al mecanismo utilizado para recibir un formulario. Por ello, las rutas pueden concentrarse en obtener datos y devolver una respuesta web.

Esta división hace posible ensayar el recorrido principal sin un navegador. El resultado de la compra se representa actualmente mediante un diccionario temporal, una solución sencilla para el flujo implementado. Su alcance, sin embargo, debe describirse con precisión: ese resultado no constituye una orden persistida ni demuestra por sí mismo la acreditación de un pago.

3. Diseño orientado a las pruebas

3.1. Dependencias sustituibles y pytest

La posibilidad de verificar comportamientos sin depender de servicios externos influyó en la organización del código. El caso de uso se relaciona con el correo a través de un protocolo, NotificadorCompra, y no mediante una llamada directa a smtplib. Esa interfaz permite reemplazar la implementación SMTP por un doble durante las pruebas, y observar tanto el envío esperado como las situaciones de error.

Con pytest se combinan funciones del dominio, fixtures, mocks y pruebas de rutas con TestClient. La ubicación configurable de SQLite permite utilizar archivos temporales para las cuentas, de manera que los escenarios no compartan datos persistentes. Este aislamiento favorece el trabajo guiado por pruebas: reduce el tiempo de ejecución, permite repetir escenarios y facilita localizar el cambio que produjo una regresión. Se trata de decisiones que apoyan TDD, aunque por sí solas no prueban el orden histórico de cada ciclo Red–Green–Refactor.

4. Datos, autenticación y notificaciones

4.1. Persistencia local con SQLite

Para el registro de usuarios se utiliza SQLite mediante el módulo sqlite3 de Python. Dado el alcance local de la aplicación, un archivo de base de datos permite conservar las cuentas sin desplegar un servicio adicional. La tabla de usuarios incluye un identificador, un correo único, el hash de contraseña y la fecha de creación; las consultas parametrizadas separan los valores recibidos de las instrucciones SQL.

La ruta configurable de la base facilita la ejecución de pruebas con datos aislados, y la restricción de unicidad del correo evita duplicar cuentas con la misma dirección. El almacenamiento implementado se limita a usuarios: las compras y los pagos no quedan registrados de manera durable. Esta distinción es importante para no atribuir a la aplicación un historial de transacciones que todavía no posee.

4.2. Contraseñas y gestión de sesiones

La autenticación almacena hashes Argon2 mediante pwdlib en lugar de contraseñas en texto claro. Para reconocer al visitante entre solicitudes se utiliza un token firmado con HMAC y vencimiento,

conservado en una cookie de sesión. De este modo, las rutas pueden restringir el acceso a la compra sin trasladar las credenciales del usuario a cada formulario.

La firma permite detectar modificaciones del token y la consulta del identificador lo vincula con una cuenta existente. La cookie utiliza HttpOnly y SameSite=Lax, además de Secure cuando la conexión se realiza por HTTPS. Si no se configura una clave de sesión estable, el servidor genera una clave temporal que invalida las sesiones al reiniciarse; es una limitación conocida del esquema actual.

4.3. Confirmaciones mediante SMTP

El envío de confirmaciones se encapsula en NotificadorSMTP, ubicado en infraestructura. Esta decisión mantiene separado el transporte del mensaje del caso de uso de compra: la aplicación solicita una notificación, mientras que el adaptador se ocupa de construir el correo, autenticarse y comunicarse con el servidor SMTP. La configuración se obtiene del entorno y no se incorpora al código fuente.

El uso de STARTTLS con verificación de certificados y un tiempo máximo de espera proporciona controles básicos para la comunicación. Los fallos se traducen a ErrorNotificacion, por lo que el caso de uso puede informar el problema sin exponer detalles sensibles. Durante las pruebas, el transporte se sustituye por mocks. La aceptación del mensaje por SMTP no garantiza su lectura ni su llegada a la bandeja de entrada, y el fallo de correo no elimina el resultado temporal de compra ya construido.

5. Alcance del pago con Mercado Pago

El pago con tarjeta se organiza como un flujo local de demostración, separado del formulario principal de compra. Antes de abrir la pantalla simulada, el servidor valida los datos del pedido y las reglas de disponibilidad; la pantalla presenta el resumen y solicita los datos de tarjeta. Esta separación mantiene las reglas de compra en el caso de uso y la interacción de tarjeta en la interfaz, lo que permite ejercitar las reglas sin depender de credenciales o servicios externos.

La pantalla comprueba en el navegador el formato del número según la marca, el nombre, el vencimiento y la longitud del código de seguridad. Si esas comprobaciones pasan, representa un resultado exitoso y envía los datos de compra a la aplicación. El flujo es demostrativo: no se conecta con la API de Mercado Pago ni obtiene una autorización externa, por lo que no constituye un pago real ni confirma que una tarjeta sea válida.

6. Convenciones de escritura del código

A lo largo del proyecto se observan nombres descriptivos y convenciones que ayudan a reconocer las responsabilidades. En Python predominan snake_case para funciones y módulos, PascalCase para clases y mayúsculas con guion bajo para constantes; las pruebas adoptan el prefijo test_. En JavaScript se utiliza camelCase. Estos patrones facilitan localizar una validación, un adaptador o un escenario de prueba a partir de su nombre.

La consistencia del vocabulario contribuye a la lectura y a las revisiones del equipo, en especial cuando una prueba debe relacionarse con una regla concreta. No obstante, no se recuperó la selección original de estilo efectuada para el TP2. Las convenciones mencionadas se fundamentan en el código observado y no se presentan como acuerdos históricos documentados; la adopción completa de PEP 8 quedaría sujeta a la definición del grupo.